

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ ПО ВЫПОЛНЕНИЮ СКВОЗНОГО ПРОЕКТА

ВЕХА 3. НЕЙРОСЕТЕВЫЕ АРХИТЕКТУРЫ И СТАТИСТИЧЕСКИЙ АНАЛИЗ УСТОЙЧИВОСТИ

Проект выполняется **КОМАНДОЙ**, в которой каждый участник имеет свою **РОЛЬ**. На этапе Вехи 3 общая командная цель - спроектировать глубокую нейронную сеть для многошагового прогнозирования последовательностей и провести стресс-тестирование пайплайна на устойчивость к изменению внешних условий.

- Роль ML Engineer: Проектирует архитектуру нейросети, настраивает циклы оптимизации весов, борется с переобучением и переводит прогноз в режим многошагового упреждения.
- Роль Data Analyst: Отвечает за разработку контура симуляции дрейфа данных, статистическую оценку падения точности моделей и расчет метрик достоверности на полигоне испытаний.
- Роль MLOps: Контролирует использование вычислительных ресурсов (GPU/RAM) при обучении сети, настраивает воспроизводимость случайных весов и оптимизирует тензорные операции.

1. ВХОДНОЙ ОРГАНИЗАЦИОННЫЙ РЕГЛАМЕНТ

Перед переходом к глубокому обучению проектная группа обязана выполнить три условия:

1. Тензоризация пайплайна. Команда подключает модуль preprocessing.py из Вехи 1, адаптируя его функции для трансформации табличных данных pandas в трехмерные тензоры PyTorch / Keras (формат: [samples, time_steps, features]).
2. Запрос вычислительных мощностей. MLOps-инженер настраивает доступ к GPU-вычислениям (через локальные CUDA-серверы или облачные платформы PaaS) для ускорения обучения рекуррентных слоев.
3. Фиксация ML-baseline. Команда выписывает лучшую ошибку (RMSE/MAPE), полученную на Вехе 2 (CatBoost), которая будет служить жестким критерием эффективности для создаваемой нейросети.

2. ЦЕЛЬ И ЗАДАЧИ ВЕХИ 3

Цель работы - проектирование, обучение и оптимизация архитектур глубокого обучения (RNN/LSTM/Conv1D) для многошагового прогнозирования временных рядов, а также проведение комплексного стресс-тестирования моделей на устойчивость к проявлениям дрейфа концептов.

Задачи вехи:

- Разработать и программно обосновать нейросетевую архитектуру (например, LSTM или одномерную сверточную сеть Conv1D) для обработки последовательностей.
- Организовать корректный процесс обучения сети: настроить функции потерь, шаг градиентного спуска, регуляризацию для борьбы с затуханием градиентов и переобучением.
- Реализовать пайплайн многошагового вывода (многошаговый прогноз спроса или нагрузки на заданный бизнесом горизонт).
- Спроектировать испытательный полигон для симуляции дрейфа данных: искусственно внедрить в валидационную выборку структурные сдвиги (смену тренда, падение амплитуды сезонности).
- Провести статистический аудит устойчивости модели и зафиксировать скорость деградации метрик качества при изменении свойств ряда.

3. ПОШАГОВЫЙ ХОД ВЫПОЛНЕНИЯ РАБОТЫ ДЛЯ СТУДЕНТОВ

Этап 1. Конструирование и обучение нейросети

- **Действия студентов:** используя фреймворк глубокого обучения (предпочтительно PyTorch), описать класс нейронной сети, содержащий рекуррентные слои (LSTM/GRU) или одномерные свертки (Conv1D). Написать кастомный класс Dataset и DataLoader для генерации батчей методом скользящего окна. Настроить функцию потерь и оптимизатор (Adam/RMSprop). Запустить цикл обучения (*training loop*) с контролем графиков функции потерь на обучающей и валидационной выборках по эпохам. Применить технику *EarlyStopping* для предотвращения переобучения. Настроить инференс модели для выдачи многошагового прогноза (на 1 час вперед для T1 или на 14 дней вперед для Спортмастера).

- **Зона ответственности ролей:** *ML Engineer* полностью кодирует архитектуру сети и циклы обучения; *MLOps* контролирует профилирование памяти GPU и исключает утечки тензоров из кэша.

Этап 2. Симуляция дрейфа и стресс-тестирование

- **Действия студентов:** спроектировать контур контроля достоверности модели. Студенты берут отложенный тестовый датасет и искусственно модифицируют его, симулируя реальные аварийные ситуации индустрии: для T1 - резкий скачок базового уровня нагрузки CPU на 40% (смена математического ожидания), для Спортмастера - внезапное исчезновение недельной сезонности из-за внешних форс-мажоров. Прогнать обученную нейросеть (и бустинг из Вехи 2) через измененные данные. Рассчитать скользящую ошибку прогноза и построить график деградации точности. Зафиксировать критическую точку (количество временных шагов), после которой модель полностью теряет прогностическую силу.

- **Зона ответственности ролей:** *Data Analyst* проектирует математические параметры сдвигов данных, проводит статистический бутстрап-анализ устойчивости ошибок и формирует аналитический отчет о границах применимости модели.

4. РЕГЛАМЕНТ ИСПОЛЬЗОВАНИЯ ГЕНЕРАТИВНОГО ИИ (LLM-КАНОН)

- **РАЗРЕШЕНО:** Использовать LLM как синтаксический автодополнитель кода для написания стандартных классов DataLoader в PyTorch, генерации циклов зануления градиентов (`optimizer.zero_grad()`) и отрисовки графиков обучения по эпохам через `matplotlib`.

- **ЗАПРЕЩЕНО:** Делегировать ИИ подбор архитектуры слоев и объяснение причин переобучения сети. Студент обязан самостоятельно обосновать выбор количества скрытых нейронов, размера батча и содержательно описать в отчете, почему нейросеть оказалась более (или менее) устойчивой к дрейфу данных по сравнению с бустингом.

5. КРИТЕРИИ ОЦЕНИВАНИЯ ВЕХИ 3 (РУБРИКАТОР ПРИЕМКИ)

- **9–10 баллов («Отлично» / Уровень Проектировщика):**

- Нейросеть (LSTM/Conv1D) написана корректно, циклы обучения и валидации полностью разделены, нет эффекта переобучения.

- Реализован и обоснован алгоритм многошагового инференса (многошаговый прогноз вперед).

- Спроектирован полноценный стресс-тест, математически корректно симулирующий дрейф концептов (*concept drift*).

- Построены графики деградации качества моделей, студент на устной защите свободно владеет формулами рекуррентных вентилей.

- **6–8 баллов («Хорошо» / Уровень Самостоятельного разработчика):**

- Нейросеть успешно обучена, графики лосса по эпохам стабильно снижаются, ошибка на тесте зафиксирована.

- Многошаговый прогноз реализован по простейшей рекурсивной схеме (когда каждый новый шаг строится на основе предыдущего предсказания), что ведет к быстрому накоплению ошибки.

- Анализ устойчивости проведен поверхностно: студент просто добавил случайный гауссов шум в данные, вместо симуляции реального индустриального изменения тренда или сезонности.

- **4–5 баллов («Удовлетворительно»):**

- **Нижний блокирующий порог.** Студент скопировал из сети код простейшей полносвязной (Linear/Dense) сети или базовой RNN и запустил ее обучение на датасете временного ряда.

- Графики обучения показывают жесткий оверфиттинг (переобучение), или модель выдает тривиальный сдвинутый на один шаг назад прогноз (модель-эхо). Анализ дрейфа данных отсутствует полностью.

- **0–3 балла («Неудовлетворительно»):** Код сети выдает ошибки размерности тензоров, обучение не сходится, или обнаружен тотальный плагиат текста отчета.